
Workshop in Autonomous Systems Simulation

Autonomous Quadcopter Drone Interceptor

A deterministic, classically-architected guidance, navigation
and control stack for counter-UAS interception in MuJoCo

Dor Varsulker

BSc Computer Science

Project submission — August 2026

Abstract

This project delivers a physically honest simulation of an autonomous counter-drone interceptor: a 1 kg quadcopter that must detect, track and physically collide with a second, possibly evasive, flying target using only noisy, delayed range-and-angle measurements. It is built as a six-stage classical hierarchical pipeline — MuJoCo plant and sensor model, Extended Kalman Filter, a lag-aware Optimal Guidance Law derived from Zero-Effort-Miss, a command limiter, a 50 Hz / 400 Hz dual-loop flight controller and a motor mixer — in which each stage reads only its immediate predecessor's published output. Deep reinforcement learning was evaluated and deliberately rejected in favour of determinism, valid physics and explainability. Validation is quantitative and reproducible: six graded KPIs measured across 22 named scenarios and a seeded 100-trial Monte-Carlo batch. The delivered system achieves a 95% mission success rate, intercepts targets up to 89.7 km/h, records a median miss distance of 0.024 m and passes 221 automated tests. One KPI — command saturation on sub-two-second high-speed engagements — is reported as failing at 77% compliance rather than tuned away, and is characterised alongside two further physical limits.

Contents

1. Introduction	3
1.1 Objectives and scope	3
2. Design rationale	4
3. System architecture	5
3.1 Data contracts	5
3.2 Scheduling and orchestration	5
3.3 Frames and conventions	6
4. Subsystem implementation	7
4.1 Stage 1 — Simulation	7
4.2 Stage 2 — Estimation (EKF)	7
4.3 Stage 3 — Guidance (OGL)	8
4.4 Stage 4 — Command limiter	8
4.5 Stage 5 — Flight control (dual loop)	9
4.6 Stage 6 — Motor mixer	9
5. Engineering methodology	10
6. Validation methodology	11
7. Results	12
7.1 Monte-Carlo breakdown	12
7.2 Named scenario suites	12
7.3 Recorded engagements	13
8. Discussion	15
9. Known limitations	16
10. Conclusions	17
11. Running and reproducing	18
11.1 Generating runs, then viewing them	18
11.2 Output artefacts	19
Appendix — Repository map and key parameters	20

1. Introduction

Small multirotor UAVs are cheap, fast and hard to stop. The defensive response of interest here is kinetic: a second quadcopter that autonomously flies to a physical collision with the intruder. The problem is harder than it sounds, because the interceptor is denied almost everything a naive solution assumes. It does not know where the target is — it receives range, azimuth and elevation readings that are noisy, biased, quantised, sampled at a finite rate and delivered late. It cannot accelerate sideways on demand, since a quadcopter's thrust points along its own body axis and all horizontal manoeuvring requires first tilting the airframe, which takes time. And it cannot simply chase: against a target at highway speed, aiming where the target is guarantees arriving where it was.

This project builds the whole problem and the whole solution in simulation. The airframe, sensor, wind and threat all live inside MuJoCo, driven by a Python guidance, navigation and control (GNC) stack. The simulation is not a demonstration harness bolted onto an algorithm — it is the experimental apparatus, and its fidelity and reproducibility are treated as engineering requirements rather than conveniences.

1.1 Objectives and scope

The system is specified against a design review whose acceptance table defines six graded KPIs (Section 6). Beyond those numbers, three objectives shaped every decision: **explainability**, so every command traces to closed-form mathematics a reviewer can audit; **determinism**, so an identical seed and configuration produce a byte-identical run log on any machine; and **physical honesty**, so actuator limits, sensor noise, latency, attitude lag and wind are modelled rather than assumed away, and exhausted control authority is reported rather than hidden.

The scope is one interceptor against one non-cooperative target in an obstacle-free volume, engaged from rest. Four things are explicitly out of scope, and the codebase rejects them rather than leaving them as tempting shortcuts: no learned guidance or control; no guidance law other than OGL (a scenario naming another law is rejected at load time rather than silently defaulting); no ground-truth shortcuts, so estimation, guidance and control may never read the simulator's true target state; and no multi-target, obstacle or onboard-vision handling.

This document is the standalone narrative of the project. It summarises four repository documents submitted alongside it, each answering a different question: the **Design Review** (what is being built and why this architecture), **PROJECT_EXPLAINED.md** (how every stage works, from scratch, with no assumed control-theory background), **ENGINEERING_STANDARDS.md** (the rules the code is built to, cited by name in source comments), **RESULTS.md** (measured performance and the commands that regenerate it) and **GETTING_STARTED.md** (install, run, troubleshoot).

2. Design rationale

The design review considered two architectures. Architecture A is the classical hierarchy: a chain of specialised, individually verifiable mathematical stages, each solving one well-posed sub-problem. Architecture B is end-to-end deep reinforcement learning, mapping sensor observations directly to motor commands.

Architecture B is genuinely attractive on raw agility, but it was rejected on four grounds that bear directly on this project's goals. Learned policies exploit simulator artefacts, becoming optimal against the physics engine's quirks rather than against physics; they are opaque, which is unacceptable for a system whose purpose is a deliberate collision; they overreact to sensor noise; and they are hard to reproduce exactly, which would forfeit the determinism guarantee that makes every result in Section 7 checkable. The trade accepted is some raw agility in exchange for determinism, valid physics and explainability — and it frames the classical stack as the interpretable analytical baseline any future learned controller would have to beat on identical seeded scenarios.

Three guidance laws were evaluated. All share the same insight from collision geometry: if the line of sight (LOS) to the target is not rotating, the two bodies are on a collision course, so guidance should drive the LOS rotation rate to zero.

Law	Principle	Outcome
Proportional Navigation (PN)	Acceleration proportional to LOS rate with a fixed navigation ratio N' .	Rejected: assumes instant turns; against a quadcopter's attitude lag it steers too gently near intercept and cannot recover.
Augmented PN (APN)	PN plus a target-acceleration term.	Rejected as the shipped law; the augmentation survives inside OGL as a gated option.
Optimal Guidance Law (OGL)	Solves the guidance optimisation <i>including</i> a first-order attitude lag, and formulates the error as Zero-Effort-Miss.	Selected and solely implemented. Anticipates the tilt lag rather than being surprised by it, and degrades to PN behaviour far from intercept.

Table 1 — Guidance-law trade study. PN and APN are not implemented.

3. System architecture

The interceptor's decision-making is a chain of six stages, each consuming only the published output of its immediate predecessor and producing its successor's input. The chain closes back into the physics and runs continuously.

```
[1 SIMULATION] -- SensorMeasurement (noisy, delayed) -->
[2 ESTIMATION] -- TargetEstimate (relative state + cov.) -->
[3 GUIDANCE]   -- AccelerationCommand (ideal) -->
[4 LIMITER]    -- LimitedAcceleration (physically safe) -->
[5a OUTER CTL] -- AttitudeReference (roll/pitch/yaw+thr.) -->
[5b INNER CTL] -- BodyWrench (torques + thrust) -->
[6 MIXER]      -- MotorCommand (4 x rotor RPM) --> back into [1]
```

The governing rule is that **no stage may reach around its input**. Guidance may not read raw sensor data; control may not read ground truth; the estimator may not consult the simulator's true target state. This is not stylistic. Each boundary is why a stage can be tested in isolation, and a violation would silently invalidate every downstream measurement — an estimator that peeked at truth would report a flattering miss distance that says nothing about the estimator. The standards document classifies such a shortcut as a defect, not an optimisation.

3.1 Data contracts

Every arrow above is a frozen dataclass: immutable (arrays are made read-only, so a consumer cannot mutate a producer's data), carrying explicit SI units in its field names, and validating shape and finiteness on construction, so a NaN or infinity raises immediately instead of propagating. The type system, not a convention, enforces the architecture.

Message	From → To	Payload
RawSensorMeasurement	Sim → Estimation	Range, LOS azimuth and elevation, timestamp, latency
TargetStateEstimate	Estimation → Guidance	Relative position / velocity / acceleration, range, LOS rate, covariance, quality
AccelerationCommand	Guidance → Limiter	The ideal, unclamped acceleration vector
LimitedAccelerationCommand	Limiter → Control	Clamped acceleration, saturation flag, magnitude removed
AttitudeReference	Outer → Inner loop	Target roll / pitch / yaw and collective thrust
BodyTorqueThrustCommand	Inner loop → Mixer	Three body torques plus thrust
MotorCommand	Mixer → Sim	Four rotor speeds in RPM

Table 2 — The seven pipeline messages: the system's entire internal interface.

3.2 Scheduling and orchestration

Stages run at different rates, as they would in a real vehicle. The physics step and the inner attitude loop run at 400 Hz (matching the 0.0025 s MuJoCo timestep); estimation follows the 100 Hz sensor cadence; guidance and the outer control loop run at 50 Hz. The rate separation between the outer and inner loops is what makes the modelled attitude lag honest, and collapsing them into one loop is forbidden. The scheduler converts these rates into integer periods in simulation steps — the 50 Hz loop fires every eighth step — so loops trigger on exact step boundaries with no accumulating floating-point drift, and it refuses to construct at all if a rate does not divide the simulation rate evenly.

The orchestrator runs that loop: read positions, update the filter if estimation is due, run guidance and the limiter if guidance is due, compute the attitude reference, run the attitude controller and mixer, step the physics, log one row. Slower stages reuse their most recent output on steps where they do not fire. Stages are *injected* rather than imported, so the identical loop runs both the pass-through stubs used by tests and the full MuJoCo / EKF / OGL stack with zero orchestrator changes. Stages also map onto six owning roles cited by number in source comments; the separation matters most at Role 5, the validation layer, which measures and reports but never tunes (Section 8).

An engagement terminates at closest approach: once the interceptor is inside the 2 m capture radius and the range begins to grow again, the intercept moment has passed and flying on would be meaningless thrashing past a target whose guidance geometry has inverted. Ending there also keeps the saturation KPI measured over the real engagement rather than diluted by a fly-by tail.

3.3 Frames and conventions

The world frame is right-handed with Z up; the body frame is X-forward, Y-left, Z-up and coincides with the world frame when level. Rotations are stored as quaternions, avoiding gimbal lock, and converted to Euler angles only where a human-readable attitude target is needed. All frame conversions and LOS geometry live in a single module, so the ground-truth kinematics, the sensor model and the estimator's own filtered LOS-rate computation cannot drift apart:

```
range = |r|    azimuth = atan2(r_y, r_x)    elevation = atan2(r_z, hypot(r_x, r_y))
azimuth_rate  = (r_x*v_y - r_y*v_x) / (r_x^2 + r_y^2)
elevation_rate = (h*v_z - r_z*dh/dt) / range^2,    h = hypot(r_x, r_y)
```

4. Subsystem implementation

4.1 Stage 1 — Simulation

This stage is the world; everything else perceives and acts through it. The scene is three MJCF files: a top-level scene fixing the timestep and an RK4 integrator, the interceptor, and the target. MuJoCo's built-in aerodynamics are switched off because wind is modelled explicitly in Python where it can be seeded and reproduced, and the interceptor's mass and inertia are asserted on load to match the constants module, so XML and code cannot silently diverge. The target is a mocap body whose pose is prescribed each step by one of five trajectory generators — static, linear, sinusoidal (evasive weaving), varying-speed (ramping to about 90 km/h), and a wind-affected wrapper — which also yields analytically exact target velocity for validation.

A rotor produces thrust and a reaction drag torque, both quadratic in speed, from which the body wrench follows:

```
thrust_i = kT * rpm_i^2      drag_i = kQ * rpm_i^2
total thrust = f_front + f_right + f_back + f_left
roll torque  = arm * (f_left - f_right)    pitch torque = arm * (f_back - f_front)
yaw torque   = reaction to the rotor drag differential
```

Two consequences drive later design. Authority is quadratic in the command, and yaw authority is about two orders of magnitude weaker than roll and pitch because it comes only from imbalancing aerodynamic drag — which is why yaw is held gently at zero, at no cost, since a quadcopter translates in any direction without turning to face it. This module is also the physical actuator boundary: speeds are clamped to 0–25,000 RPM and every clamp is reported as a saturation event.

The sensor deliberately corrupts perfect geometry into what a real tracker would produce: Gaussian noise (0.30 m on range, roughly 0.2° on the angles), optional bias and quantisation, a 100 Hz update rate against 400 Hz physics, and 0.02 s latency implemented as a real delay buffer with each sample stamped with its true age. Cleaning it up for downstream convenience is forbidden — the noise is the entire reason the estimator exists — and constructing a sensor without a seeded random source is a fatal error. Wind is a steady breeze plus Ornstein-Uhlenbeck gusts, which are smooth and temporally correlated rather than white; the whole gust series is precomputed once from its own named stream, so wind velocity is a pure function of time. Three presets exist: calm (exactly zero, bit-identical to undisturbed physics), moderate and gusty.

Ground truth stays inside Stage 1. True relative geometry, range, LOS angles and closing speed are computed in the simulation layer and used for exactly two purposes: feeding the sensors, which corrupt it, and verifying results in tests. Estimation, guidance and control never see it.

4.2 Stage 2 — Estimation (EKF)

A Kalman filter holds two imperfect sources of information — a physics-based prediction and a fresh measurement — and blends them in proportion to their uncertainties, alternating between a predict step that grows uncertainty and a correct step that shrinks it. The filter must be *extended* because the measurement is non-linear: range and two angles relate to Cartesian position through square roots and atan2 , so the EKF linearises about the current estimate using the analytic Jacobian of the measurement equations. The state is nine-dimensional and relative, with a constant-acceleration motion model:

```
x = [ position(3), velocity(3), acceleration(3) ]    (target - interceptor)
position += velocity*dt + 0.5*acceleration*dt^2    velocity += acceleration*dt
```

Four details carry most of the robustness. Angle innovations are wrapped to $(-\pi, \pi]$ so a reading crossing $\pm 180^\circ$ does not manufacture a false error. The covariance update uses the Joseph form, which keeps the uncertainty matrix symmetric and positive under floating-point rounding. Latency is compensated by predicting the filter's own state forward by the measurement's stamped age, so guidance receives an estimate valid *now* rather than 20 ms ago. And divergence fails loud: a non-finite state or a covariance trace past a configured bound raises rather than emitting garbage. The filter also publishes a quality score in $[0, 1]$. One simplification is deliberate: the interceptor's own manoeuvres are unknown to this layer and are absorbed into process noise — the choice that later gates the augmented guidance term.

4.3 Stage 3 — Guidance (OGL)

Guidance answers where and how hard to accelerate, and deliberately ignores whether the airframe can comply. It is built on Zero-Effort-Miss (ZEM): the vector by which the interceptor would miss if it never accelerated again.

```
t_go = time_to_go(r, v) # conditioned and clamped
N'   = clamp(lag_aware_nav_ratio(t_go, T), 3, 5)
ZEM  = r + v*t_go + 0.5*a*t_go^2
a_cmd = N' / t_go^2 * ZEM ; a_cmd[Z] *= 1 / (1 + b) # altitude penalty
```

Time-to-go is nominally range over closing speed, but that is singular at both ends: at launch the interceptor is at rest so the quotient diverges, and near intercept the $1/t_go^2$ factor explodes. The implementation uses the true quotient when genuinely closing, synthesises a value from a 3.5 m/s reference closing speed when closing speed is negligible (a static target or a from-rest launch), and clamps the result to $[0.05 \text{ s}, 30 \text{ s}]$.

The lag-aware navigation ratio is what makes the law *optimal* rather than merely proportional. A quadcopter's attitude response is modelled as a first-order lag $1/(Ts + 1)$ with $T = 0.2 \text{ s}$. Plain PN ignores it, steers too gently as intercept approaches, and cannot recover. Solving the optimisation with the lag included gives the classic closed-form Zarchan solution, a ratio varying with normalised time-to-go $x = t_go / T$:

$$N'(x) = 6x^2(e^{-x} - 1 + x) / (2x^3 - 6x^2 + 6x + 3 - 12xe^{-x} - 3e^{-2x})$$

For large x this tends to 3, so the law behaves like ordinary PN; for small x it grows, steering harder *in advance* to pre-empt the tilt lag. The result is clamped to $[3, 5]$. In short, it is a navigation gain that knows the airframe is sluggish.

Two further choices. The altitude penalty b (default 0.1) de-weights the vertical channel, included because the design review flags vertical overshoot as a recurring interceptor failure mode; an ablation pair with $b = 0$ found its measured effect near zero here (Section 8). And the ZEM term $0.5*a*t_go^2$ — the augmentation that would let the law anticipate a manoeuvring target — is implemented but **gated off**, because the EKF estimates *relative* acceleration, which against a non-manoevring target largely reflects the interceptor's own manoeuvring. Feeding that back would be positive feedback on its own motion. Isolating absolute target acceleration is the open work that would unlock it.

4.4 Stage 4 — Command limiter

Guidance can ask for anything. The limiter is the single place that clamps the request, and — equally important — the single place where acceleration saturation is measured. Because horizontal acceleration comes only from tilting, the horizontal cap is $g \cdot \tan(\text{max tilt})$, approximately 27.0 m/s^2 at the tuned 70° ; total commanded magnitude is separately capped at 40 m/s^2 . Every clamp is reported with a flag and the magnitude removed. Concentrating all clamping here is what makes the saturation KPI meaningful: being pinned at a limit means

control authority has been lost, and the project measures precisely how often that happens.

4.5 Stage 5 — Flight control (dual loop)

A safe acceleration is still not an input a quadcopter accepts. An outer loop at 50 Hz asks what tilt and thrust achieve the commanded acceleration; an inner loop at 400 Hz rotates the body to that tilt. The outer loop exploits differential flatness: a desired acceleration maps algebraically to attitude and thrust with no feedback tuning, because the rotors can only push along the body up-axis:

```
f      = a_cmd + [0, 0, g]          thrust = mass * |f|
roll   = atan2(-f_y, hypot(f_x, f_z)) pitch  = atan2(f_x, f_z)  # yaw = 0
```

Hovering is the degenerate case: zero commanded acceleration gives a purely vertical vector, zero tilt and thrust exactly equal to weight. This gain-free, exact mapping is much of why the system overshoots altitude so little. The inner loop realises the commanded tilt from the gyroscope; because a rate gyro measures angular velocity rather than orientation, the controller maintains its own attitude by strapdown quaternion integration, seeded level, and runs a PD law (a proportional term on attitude error, a derivative term opposing body rate) converted to torque through the inertia matrix. Roll and pitch are tuned for a natural frequency of about 17 rad/s, well inside the 400 Hz loop; yaw gains are roughly a hundred times smaller, because yaw torque comes from the weak drag differential and an aggressive gain would saturate all four motors chasing an axis that does not help interception. The loop is not faked — attitude responds at its natural speed, and that real lag is exactly what OGL anticipates.

4.6 Stage 6 — Motor mixer

The final translation inverts the rotor model: given a collective thrust and three body torques, solve the four-equation linear system for the four rotor thrusts, then invert the quadratic thrust law for rotor speeds. The mixer reads the same physical constants as the forward model, so plant physics and controller inverse cannot disagree. Speeds are clamped to 0–25,000 RPM; if the allocation would demand negative thrust from a rotor, or exceed maximum RPM, the mixer clamps and reports its own saturation event. That second saturation source is not cosmetic — Section 8 explains why counting it changed the project's headline compliance number.

5. Engineering methodology

The codebase is built to a written standards document that source comments cite by name. Six rules are load-bearing for claims made later in this report.

- **Determinism.** Identical seed plus identical configuration gives a byte-identical run log. All randomness flows through one seeded factory deriving independent *named* streams (sensor, wind), so adding a stochastic component never perturbs existing numbers, and global NumPy random state is never touched. Order-dependent processes such as the gust series are precomputed. Every run writes `run_config.json` (root seed, resolved parameters, git hash) and every batch a manifest with its master seed, so (master seed, trial count) regenerates a batch exactly. Dependencies are pinned.
- **Fail loud, never silent.** Non-finite values in any message raise on construction; EKF covariance blow-up raises rather than emitting a garbage estimate; unknown scenario keys, unknown trajectory types and any law other than OGL are rejected; a sensor with no noise profile or no seed is refused, because a conveniently clean sensor would quietly invalidate the whole estimation problem.
- **Configuration discipline.** Physical and structural facts (mass, arm length, rotor coefficients, loop rates, capture radius, KPI thresholds) are separated from tunables (EKF covariances, guidance gains, limiter bounds, PID gains, sensor profile). No module may hold a magic number, and any scenario may deep-merge overrides onto the tunables without touching code. Changing a KPI-affecting default carries an inline rationale and a full re-run of the suite.
- **Saturation must stay measurable.** The limiter is the sole clamp on requested acceleration; the mixer reports its own clamping; the KPI counts the *union*, with per-stage columns logged for attribution. Counting one stage alone would let the airframe be pinned while the metric stayed green.
- **Headless, non-interactive execution.** Nothing may open a blocking window; plotting uses matplotlib's Agg backend; the orchestrator fails loud if an interactive viewer is requested from an automated run. The one carve-out is the opt-in replay viewer, which re-renders an already-recorded log and therefore cannot influence any result.
- **Testing.** Unit tests cover components in isolation, integration tests the closed loop, scenario suites, Monte-Carlo batch and headless renderer. All are headless, non-interactive and seeded, and none may depend on wall-clock timing. Determinism itself is under test: a re-run batch must reproduce its results. **221 tests pass** in about 80 seconds, and static analysis must be clean.

6. Validation methodology

Validation is layered. Named scenarios probe specific reproducible geometries and answer “does it handle this case?”; the randomized Monte-Carlo batch samples the whole threat envelope and answers “how often does it work?”. Both drive the identical closed loop, and neither contains physics logic of its own. Success is defined by six metrics from the design review's acceptance table, each already carrying a 5% margin. A single module is the sole authority for turning a recorded run log into graded metrics; it never re-runs physics, it only measures. Vertical overshoot is computed sign-aware — above the target for a climbing intercept, below for a descending one — which corrected a defect where a descending approach counted its benign initial altitude gap as enormous overshoot.

KPI	Target	Definition
Mission success rate	$\geq 90\%$	Fraction of randomized 3D trials ending in an interception.
Miss distance R_{miss}	$\leq 1.05 \text{ m}$	Closest approach; a hit is any run within this radius.
Time-to-intercept	static $< 10 \text{ s}$, moving $< 20 \text{ s}$	Elapsed time from launch to closest approach.
Z-axis overshoot	$\leq 0.5 \text{ m}$	Sign-aware distance flown past the target's altitude.
Command saturation	$\leq 5\%$ of flight time	Fraction of the engagement pinned at a limiter <i>or</i> mixer bound.
Max target speed	$\geq 83.6 \text{ km/h}$	Fastest target still defeated.

Table 3 — The six graded KPIs and their acceptance targets.

A scenario is a small YAML file fully specifying one trial: seed, target class, trajectory type and parameters, interceptor start, time limit, optional wind preset, optional parameter overrides, and always OGL. The runner declares and drives, reusing the same generators and closed loop as everything else, and fails loud on unknown trajectory types, missing or misspelled keys, malformed vectors, a non-OGL law, or a file setting both a wind preset and an explicit wind block. The library holds 6 static and 5 linear baseline geometries, an ablation pair for the altitude-penalty study, and a stress directory of 4 sinusoidal, 3 varying-speed and 4 wind scenarios.

From a single master seed the Monte-Carlo harness draws randomized 3D engagements — geometry in a frontal cone, a weighted trajectory family, family parameters and a weighted wind preset — converts each into a validated scenario and flies it. It reports mission success as the interception fraction, per the design review's definition, and the other five KPIs as separate compliance rates broken down per family and per wind preset, so weak regimes are exposed rather than averaged away. Each trial's run seed is its index, so the batch reproduces byte-for-byte.

7. Results

Every number below was produced by the commands quoted, on the committed code, with fixed seeds; none is hand-copied from an earlier run. The reference machine is an eight-core x86-64 laptop running Linux on CPU only, Python 3.14, MuJoCo 3.10.0. Measured values are the canonical randomized batch of 100 trials at master seed 0.

KPI	Target	Measured	Verdict
Mission success rate (interception)	$\geq 90\%$	95% (95/100)	✓
Max target speed intercepted	≥ 83.6 km/h	89.7 km/h	✓
Miss distance R _{miss}	≤ 1.05 m	95% of trials; median hit 0.024 m	✓
Z-axis overshoot	≤ 0.5 m	98% compliance; median 0.002 m	✓
Time-to-intercept	static < 10 s, moving < 20 s	95% compliance — every successful intercept was inside budget	✓
Command saturation	$\leq 5\%$ of flight time	77% compliance; median 1.3%	⚠

Table 4 — Headline acceptance table. The saturation row is a reported tail failure, characterised in Section 9.

75 of the 100 trials passed every KPI simultaneously; the gap between that and the 95% mission-success figure is almost entirely the saturation tail on very short, very fast engagements. Mission success is defined as interception, per the design review, and the remaining KPIs are reported as separate compliance rates — so a two-second high-speed intercept that transiently exceeds the saturation budget is recorded as a success with a documented caveat, neither a hidden failure nor a silent pass.

7.1 Monte-Carlo breakdown

Target family	Interception	Wind preset	Interception
static	21/21 (100%)	calm	58/62 (94%)
linear	32/32 (100%)	moderate	23/23 (100%)
sinusoidal (evasive)	34/35 (97%)	gusty	14/15 (93%)
varying-speed (to 90 km/h)	8/12 (67%)		

Tables 5 and 6 — Interception by target family and by wind preset over the canonical 100-trial batch.

Two findings stand out. Interception is essentially flat across calm, moderate and gusty conditions, confirming wind robustness: with the modelled lumped drag the disturbance acts as a slow bias the dual-loop controller absorbs. And all five misses are fast, off-axis varying-speed or sinusoidal draws, not wind failures. The degradation edge is geometric and kinematic, not environmental.

7.2 Named scenario suites

All eleven baseline static and linear scenarios pass every KPI (about 21 s).

Scenario	Class	R _{miss} (m)	t _{int} (s)	Z-over (m)	Sat %	Pass
linear_approaching	moving	0.032	4.78	0.021	1.1	✓
linear_climbing	moving	0.011	4.13	0.001	0.0	✓

Scenario	Class	R_miss (m)	t_int (s)	Z-over (m)	Sat %	Pass
linear_crossing	moving	0.016	5.23	0.004	0.3	✓
linear_diagonal	moving	0.008	5.62	0.005	0.2	✓
linear_receding	moving	0.014	4.27	0.010	0.5	✓
static_descend	static	0.013	2.28	0.000	1.3	✓
static_diagonal	static	0.037	4.59	0.006	1.2	✓
static_far	static	0.003	7.70	0.000	0.0	✓
static_high	static	0.004	3.71	0.001	0.0	✓
static_lateral	static	0.013	2.23	0.037	2.7	✓
static_near_level	static	0.009	2.71	0.004	1.5	✓

Table 7 — Baseline scenario suite, 11/11 passing.

All eleven stress scenarios intercept; nine pass every KPI (about 14 s). The two failing rows are saturation breaches, not misses — every closest approach is inside the 1.05 m hit radius.

Scenario	R_miss (m)	t_int (s)	Z-over (m)	Sat %	Max spd (km/h)	Pass
sinusoidal_3d_spiral	0.058	2.86	0.039	0.5	13.3	✓
sinusoidal_fast_juke	0.654	4.37	0.373	10.3	17.6	✗
sinusoidal_lateral_weave	0.041	5.76	0.034	0.8	9.0	✓
sinusoidal_vertical_bob	0.177	4.39	0.000	1.5	9.8	✓
varying_speed_crossing_84kmh	0.325	2.06	0.000	5.8	78.0	✗
varying_speed_headon_90kmh	0.112	2.08	0.000	3.4	72.2	✓
varying_speed_quartering_86kmh	0.127	2.11	0.000	3.0	70.3	✓
wind_evasive_moderate	0.079	4.81	0.000	1.3	13.8	✓
wind_linear_gusty	0.017	4.19	0.000	1.2	7.2	✓
wind_static_gusty	0.032	4.76	0.011	0.9	0.0	✓
wind_static_moderate	0.005	5.74	0.002	0.0	0.0	✓

Table 8 — Stress suite: evasive, high-speed and wind probes.

The maximum-speed column reports the target's speed *at the intercept instant*, which is why the 90 km/h ramp scenarios show values in the low seventies: they are caught before reaching peak speed. That is the point of leading a target rather than chasing it, and it is visible directly in the data.

7.3 Recorded engagements

The figures below are replay-viewer captures of recorded runs, showing the interceptor trajectory in cyan and the target trajectory in yellow. They are re-renders of the logged history and read no live ground truth, so they cannot affect any result. The submission ships no results/directory, so these views are reproduced by generating the runs first and then replaying any of them with `python scripts/replay.py results/<run_dir>` (Section 11.1).

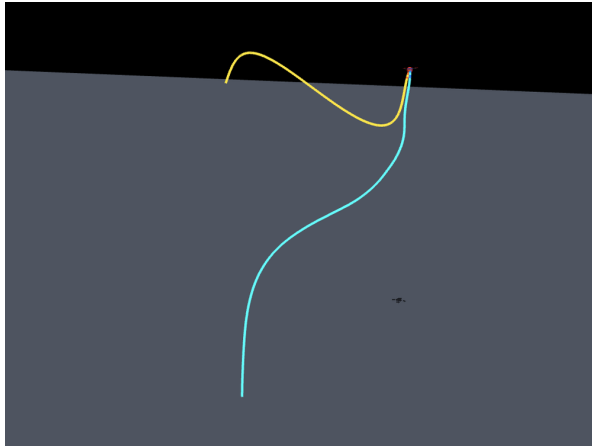


Figure 1 — *sinusoidal_3d_spiral*. An evasive spiralling target. The interceptor climbs and curves onto a lead course rather than tracking the weave.

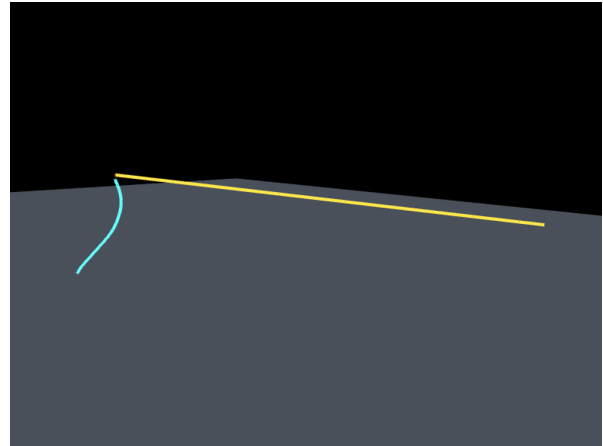


Figure 2 — *varying_speed_crossing_84kmh*. A high-speed crossing target at 84 km/h. The engagement is short and nearly straight — the regime that drives limitation L1.

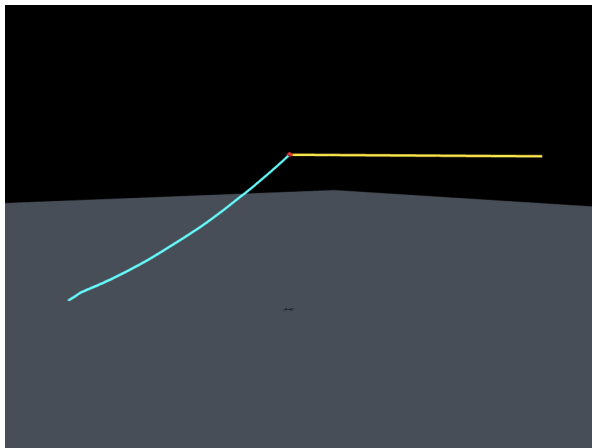


Figure 3 — *wind_linear_gusty*. A linear target under gusty wind. The trajectory stays smooth: gusts act as a slow bias the dual loop absorbs.

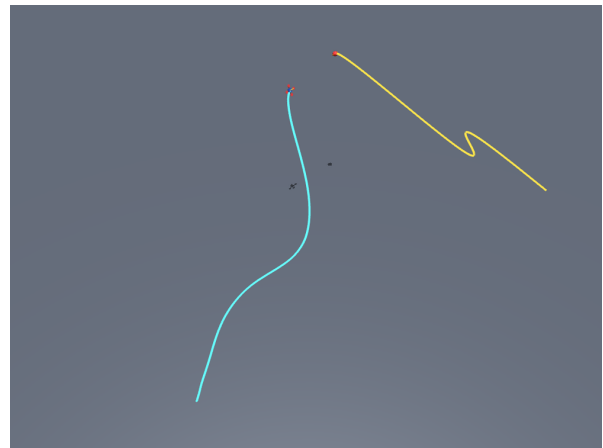


Figure 4 — *wind_evasive_moderate*. An evasive target under moderate wind, viewed from above. The interceptor leads the target instead of chasing its weave.

8. Discussion

The single largest performance driver was not the guidance law but how much horizontal authority the limiter was permitted to release. Since horizontal acceleration is $g \cdot \tan(\text{tilt})$, the maximum tilt angle sets the cap directly, and it was raised in stages, each driven by a measurement rather than a guess.

Max tilt	Authority	What the measurement showed
35° (original)	~6.87 m/s ²	Cross-range dashes hit the cap constantly.
45°	9.81 m/s ² (1 g)	Weaving and 90 km/h targets still clamped for 15-30% of a short engagement — the dominant saturation-KPI failure.
60°	~17 m/s ²	Randomized mission success rose from roughly 57% to 93%; the total-magnitude cap moved from 30 to 40 m/s ² alongside it.
70° (shipped)	~27 m/s ²	Viable once the outer loop's exact thrust projection removed the altitude overshoot that had made a larger tilt unattractive.

Table 9 — Tilt-authority tuning history.

The final step is paired with an inner-loop angular-acceleration clamp at 70 rad/s², the hover-feasible ceiling, so a large commanded slew is rate-limited rather than demanding infeasible torque. With a thrust capacity on the order of 250 m/s², these limits represent genuinely released authority rather than saturation displaced into the motors. A second decision runs the other way: the reference closing speed used to synthesise time-to-go at launch was reduced from 5.0 to 3.5 m/s, because a higher value saturated the motors through the first fifth of the flight while a lower one failed to reach the farthest static target in budget. It is a documented trade-off, not an optimum.

A validation finding that made the metric stricter. The saturation KPI originally counted only the command limiter. The validation role found this systematically undercounted: the motor mixer could be clamping rotor allocations — the airframe genuinely pinned — while the reported metric stayed green. The response was to make the metric stricter rather than the target looser, so saturation now counts the union of limiter and mixer events with per-stage columns kept for attribution. The 77% compliance in Table 4 is the number *after* that correction, and is substantially worse than the pre-fix figure. Reporting the worse, truer number is the intended behaviour of the separation between the measuring role and the tuning roles.

What the ablation showed. The guidance altitude-penalty knob b was included on the design review's warning that vertical overshoot is a common interceptor failure mode. The dedicated ablation pair showed its measured effect here to be near zero, because the differential-flatness outer loop already maps commanded acceleration to attitude and thrust exactly, leaving little vertical error to penalise — visible in the 0.002 m median overshoot of Table 4. This is a useful negative result: the problem the knob was designed to solve had already been solved structurally, by the choice of controller.

9. Known limitations

Three limits remain, each characterised and reproducible from the seeds given rather than tuned away, and each filed as a finding against an owning role.

- **L1 — Command saturation on sub-two-second high-speed intercepts.** 23% of randomized trials exceed the 5%-of-flight-time budget. The regime is narrow: an 85+ km/h crossing or quartering target engaged from rest and closed in roughly two seconds (Figure 2). The interceptor still hits, but spends a large share of that very short flight pinned against the tilt cap or the mixer's allocation limit. Closing this needs adaptive command authority or launch-shaping in guidance — a logic change, deliberately outside parameter-only tuning.
- **L2 — Fast off-axis targets are the physical degradation edge.** The varying-speed family carries four of the batch's five misses. An interceptor launched from rest cannot always lead a strongly crossing target above roughly 85 km/h; this is engagement geometry, not an implementation defect. The candidate fix is the gated augmented-ZEM term, which stays gated until absolute target acceleration can be isolated.
- **L3 — Far-static engagements are tight against the 10 s budget.** Static targets beyond roughly 12 m are intercepted, but the time metric can slip against a deliberately strict budget. In the canonical batch every successful intercept was inside budget; the five time failures are exactly the five misses, since no intercept means no intercept time.

10. Conclusions

The delivered system meets the acceptance table on five of six KPIs and reports the sixth honestly. Over 100 seeded randomized 3D trials it achieves 95% mission success with a median miss distance of 0.024 m, intercepts targets up to 89.7 km/h against an 83.6 km/h requirement, keeps vertical overshoot compliant in 98% of trials, passes all eleven baseline scenarios on every KPI, intercepts in all eleven stress scenarios, and holds interception essentially flat across calm, moderate and gusty wind — backed by 221 passing headless tests and a determinism guarantee enforced by one of them.

Three architectural conclusions carry forward. The classical hierarchy was the right choice for this brief: every result is explicable in terms of a specific stage, and the tuning history in Section 8 was only possible because authority limits live in exactly one measurable place. Structural fixes beat tuning knobs — the differential-flatness outer loop eliminated the altitude overshoot a guidance-side penalty term had been added to manage. And strict measurement is worth more than a flattering headline: the saturation KPI was made harsher mid-project, and the resulting failing row is the most informative line in the acceptance table, because it points precisely at the regime where the airframe runs out of authority.

The limits recorded in Section 9 are of a piece with that: each is characterised, reproducible from a stated seed, and attributed to a specific stage rather than smoothed into the headline figures.

11. Running and reproducing

The project needs Python 3.11+, roughly 1 GB of disk and CPU only — no GPU, no CUDA, and no separate MuJoCo installation, since the pip wheel bundles its native libraries. A display is needed only for the replay viewer; every other command is headless. Dependency versions are pinned because determinism is the core guarantee.

```
# Install and verify
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
python scripts/check_env.py          # Python, imports, MuJoCo, one off-screen frame

# A single guided interception                                     (~5 s)
python scripts/run_intercept.py --target 8 3 6 --seconds 9

# Reproduce the published results
python scripts/run_scenarios.py scenarios/ --report \
    --results-dir results/scenarios                                     (~21 s)
python scripts/run_scenarios.py scenarios/stress \
    --results-dir results/stress                                       (~14 s)
python scripts/run_montecarlo.py --trials 100 --seed 0 --report \
    --results-dir results/montecarlo                                   (~1 min 50 s)

# Tests and lint
pytest                               # 221 tests, ~80 s
pytest -m "not mujoco"               # skip tests needing an off-screen GL context
ruff check src tests scripts
```

11.1 Generating runs, then viewing them

The `results/` directory is generated output. It is git-ignored and is **not included in the submission**, so a fresh checkout contains no runs and there is nothing to replay until something has been flown. Run a command from the block above first; each one creates the run directories the viewer then reads. Because the runs are seeded, what you generate is what this report describes — the same trajectories, the same numbers.

Run this	Creates
<code>run_intercept.py --target 8 3 6 --seconds 9</code>	<code>results/intercept/</code> — one run, ~5 s
<code>run_scenarios.py scenarios/ --results-dir results/scenarios</code>	<code>results/scenarios/<scenario>/</code> — the 11 runs of Table 7
<code>run_scenarios.py scenarios/stress --results-dir results/stress</code>	<code>results/stress/<scenario>/</code> — the 11 runs of Table 8
<code>run_montecarlo.py --trials 100 --seed 0 --report --results-dir results/montecarlo</code>	<code>results/montecarlo/trial_XXXX/</code> — one directory per trial

Table 10 — What each command leaves on disk for the viewer to read.

Then point the replay viewer at any one of those run directories:

```
python scripts/replay.py results/<target_run_dir>

python scripts/replay.py results/intercept          # top isometric view
python scripts/replay.py results/stress/sinusoidal_3d_spiral # a stress scenario
python scripts/replay.py results/montecarlo/trial_0007    # one Monte-Carlo trial

--view interceptor    chase camera    --speed 0.5    slow motion
--loop                replay continuously
```

Point it at a *leaf* run directory — one containing `run_log.csv` and `run_config.json` — not at a suite root such as `results/stress`, which only holds the per-scenario folders. A complaint about a missing log is almost always one of those two causes: the run has not been generated yet, or the path is one level too high.

The viewer re-renders the recorded log rather than re-simulating, which is why it is the one deliberately interactive tool in an otherwise headless project: it reads no live ground truth and cannot change any result. Both drones draw trajectory trails (interceptor cyan, target yellow), and without `--loop` the view freezes on the intercept frame — which is how the captures in Section 7.3 were taken. Skip it entirely on a headless machine and use the CSV logs and generated plots instead.

11.2 Output artefacts

Each run directory holds `run_log.csv` (one row per simulation step: time, interceptor pose, target position, estimated range, commanded acceleration magnitude, union and per-stage saturation flags, four rotor speeds) and `run_config.json` (root seed, resolved parameters, git hash, resolved scenario spec). Suite and batch runs add `kpi_summary.csv`, per-scenario diagnostic plots, `batch_kpis.csv` (one row per trial), `batch_manifest.json` (master seed, git hash, KPI targets, committed tuning, headline verdicts) and `batch_distributions.png`. The whole `results/` directory is generated output and is git-ignored.

Portability note. Small differences between floating-point libraries across machines can move the final digits of a miss distance. The pass/fail verdicts and headline rates are stable; byte-identical reproduction is guaranteed for a given machine and environment, and is asserted by an integration test.

Appendix — Repository map and key parameters

```

Autonomous-Quadcopter-Drone-Interceptor/
|- docs/           Design review, PROJECT_EXPLAINED, standards, results, guide
|- models/         MuJoCo world: scene.xml, quadcopter.xml, target.xml
|- scenarios/       11 baseline YAML + ablation/ + stress/ (11 probes)
|- scripts/         check_env, run_intercept, run_scenarios, run_montecarlo,
|                   run_stub_pipeline, replay
|- src/interceptor/
|   |- config/      constants.py (physical) + params.py (tunable)
|   |- common/      types, frames, rng, logging, guards
|   |- simulation/   [1] mujoco_plant, actuators, sensors/, trajectories/,
|   |               kinematics, wind, rendering, interfaces, stubs
|   |- estimation/   [2] ekf
|   |- guidance/     [3] ogl, zem, time_to_go
|   |- control/      [4,5] command_limiter, outer_loop, inner_loop, motor_mixer
|   |- pipeline/     [6] orchestrator, scheduler
|   +- analysis/     [5] kpis, scenarios, montecarlo, reporting
|- tests/           unit/ + integration/ -- headless, seeded (221 tests)
+- results/         Generated logs, snapshots, reports (git-ignored)

```

Quantity	Value	Class
Interceptor mass / arm length	1.0 kg / 0.15 m	constant
Rotor speed range	0 – 25,000 RPM	constant
Simulation timestep / rate	0.0025 s / 400 Hz (RK4)	constant
Outer, inner, estimation rates	50 Hz, 400 Hz, 100 Hz	constant
Attitude lag time constant T	0.2 s	constant
Capture radius	2.0 m	constant
Maximum tilt	70° (1.2217 rad) → ~27.0 m/s ² horizontal	tunable
Maximum commanded acceleration	40 m/s ²	tunable
Inner-loop angular acceleration clamp	70 rad/s ²	tunable
Navigation ratio bounds / t _{go} clamp	N' in [3, 5] / [0.05 s, 30 s]	tunable
Reference closing speed (t _{go})	3.5 m/s	tunable
Guidance altitude penalty b	0.1	tunable
Sensor noise, rate and latency	0.30 m / ~0.2°, 100 Hz, 0.02 s	tunable
Augmented ZEM term	implemented, gated off	tunable

Table 11 — Selected constants and tunables. Physical constants describe the airframe and the acceptance contract; tunables are the values engineering sweeps. A full glossary of terms is in `PROJECT_EXPLAINED.md`.